[Projet] Colloc' gestion de colocation avec Flutter et Supabase

Prérequis : TP Flutter et TP Supabase. Ici, **aucun code n'est fourni**, seulement un cahier des charges.

Le produit : une application mobile où des colocataires gèrent leur vie commune : dépenses partagées avec calcul des soldes, tâches ménagères, liste de courses collaborative et chat de la coloc.

1. Cahier des charges

Fonctionnalités obligatoires

Comptes et colocations

- Inscription, connexion, déconnexion, profil (pseudo, avatar).
- Un utilisateur peut **créer** une colocation ou la **rejoindre** avec un code d'invitation.
- Un utilisateur peut appartenir à **plusieurs** colocations et basculer de l'une à l'autre.
- Rôles : `admin` (le créateur) et `membre`. Seul un admin peut retirer un membre ou régénérer le code d'invitation.

Dépenses partagées

- Ajouter une dépense : montant, payeur, description, date, **répartition entre certains membres** (parts égales, ou parts personnalisées), photo du ticket (Storage).
- Écran « Soldes » : qui doit combien à qui, avec le **minimum de remboursements** à effectuer.
- Marquer un remboursement comme effectué.

Tâches ménagères

- Tâches récurrentes (quotidienne, hebdomadaire) avec **rotation automatique** entre les membres.
- Marquer une tâche comme faite, historique, indicateur de retard.

Courses

- Liste partagée **en temps réel** : ajouter, cocher, supprimer. Voir qui a ajouté ou coché quoi.

Chat

- Messages de la coloc en temps réel, avec chargement progressif de l'historique et indicateur de nouveaux messages.

Exigences transverses

- **Sécurité par RLS exclusivement** : aucune confiance accordée au client.
- Gestion des erreurs et de l'état de chargement sur tous les écrans, état « liste vide » soigné.
- **Mode hors connexion minimal** : l'application ne plante pas sans réseau et l'indique clairement.
- Interface adaptée au thème clair et sombre.

Notifications locales de rappel de tâche, statistiques en graphiques, export CSV des dépenses, invitation par lien, cache local (lecture hors connexion), tests d'intégration.

2. Règles du jeu

1. **Pas de code fourni, pas de modèle de projet.** Seuls `flutter create` et le package `supabase_flutter` sont autorisés comme point de départ.
2. **Un dépôt Git** avec des commits **réguliers, par personne et par fonctionnalité**. L'historique fait partie de la notation.
3. **Un journal de bord** (fichier `JOURNAL.md` à la racine) : à chaque séance, chacun note ce qu'il a fait, une difficulté rencontrée et comment il l'a résolue.
4. **Des décisions d'architecture documentées** (fichier `DECISIONS.md`) : pour chaque choix important (gestion d'état, structure des dossiers, modèle de données), 3 à 5 lignes : *options envisagées, choix, raison*.
5. **Les clés** : la clé `service_role` ne doit apparaître **nulle part** dans le dépôt ni dans l'application. Un seul manquement entraîne un malus.
6. **Aucun filtre de sécurité côté Flutter** : toute restriction d'accès doit exister en base (RLS).

3. Planning et jalons

1 : Conception, aucun code Flutter

Livrables à valider avant de coder :

- **Schéma de base de données** (diagramme + script SQL) : des tables, des clés et des relations. Questions à trancher sur papier :
  - Comment représenter « un utilisateur appartient à une coloc, avec un rôle » ?
  - Comment représenter une dépense **et** sa répartition entre plusieurs personnes ? (Une seule table suffit-elle ?)
  - Stocke-t-on les montants en décimal ou en centimes entiers ? Pourquoi ?
  - Où stocker la rotation des tâches ?
- **Maquettes papier** de 5 écrans clés et **graphe de navigation**.
- **Matrice d'autorisations** : un tableau « qui peut lire / créer / modifier / supprimer quoi » (non-membre, membre, admin, auteur de la ligne).
- **Découpage en tâches** réparties entre les membres.

> **Jalon 1 :** le schéma : valide, ou renvoie à la conception.

2 : Fondations

- Création des tables, **relations, contraintes** (`CHECK`, `NOT NULL`, `UNIQUE`) et **toutes les policies RLS** de la matrice.
- Structure du projet Flutter, couche d'accès aux données séparée de l'interface, modèles Dart.
- Authentification complète et redirection automatique selon l'état de connexion.
- Création et adhésion à une coloc via code d'invitation.

> **Jalon 2 :** démonstration en direct avec **deux comptes sur deux appareils** : A crée une coloc, B la rejoint avec le code. Puis, avec la clé anon et le token de B, une requête directe (`curl`) vers les données d'une coloc **dont B ne fait pas partie** doit être refusée ou vide.

3 : Dépenses et soldes

- Écrans de saisie et de liste des dépenses, répartition personnalisée, photo du ticket.
- **Calcul des soldes** : le groupe décide s'il se fait en SQL (vue ou fonction RPC) ou en Dart, et **justifie** son choix dans `DECISIONS.md`.
- Algorithme de **simplification des dettes** (minimum de transferts).
- Écran « Soldes » et remboursements.

> **Jalon 3 :** jeu de test imposé par exemple 4 personnes, 5 dépenses avec des parts inégales). tu dois **prédire les soldes sur papier**, puis montrer que l'application donne le même résultat. La somme des soldes doit valoir exactement 0.

4 : Temps réel, tâches, courses, chat

- Liste de courses et chat en temps réel, pagination de l'historique.
- Tâches récurrentes avec rotation (logique à placer en base ou côté application, au choix, à justifier).
- Gestion propre des abonnements Realtime (ouverture et fermeture à la sortie de l'écran, reconnexion).

> **Jalon 4 :** démonstration de la synchronisation entre deux appareils, puis **coupure du réseau** sur l'un d'eux : l'application doit réagir proprement et se resynchroniser au retour de la connexion.

5 : Finitions, tests, soutenance

- Gestion des erreurs, états vides, thèmes, retours utilisateur.
- **Au minimum 5 tests automatisés** (tests unitaires du calcul des soldes, tests de widgets sur un formulaire).
- Audit de sécurité croisé : chaque étudiant **attaque la base d'un autre** (avec sa clé anon publique et un compte de test) et rédige un rapport d'une demi-page.
- Soutenance (voir §5).

4. Points techniques à anticiper (sans solution fournie)

Ces questions guident la réflexion. Les groupes y répondent dans `DECISIONS.md`.

- **RLS multi-utilisateurs :** pour qu'un membre voie les données de sa coloc, la policy doit consulter la table des appartenances. Que se passe-t-il quand la table des membres a elle-même une policy qui consulte… la table des membres ? Comment contourner proprement ce problème ?
- **Invitation :** comment un utilisateur qui n'est **pas encore membre** peut-il rejoindre une coloc dont il ne peut pas lire les données ? Quel mécanisme garde cela sûr ?
- **Intégrité :** comment empêcher qu'une dépense référence un payeur qui n'est pas membre de la coloc ? (Contraintes, triggers, policies ?)
- **Arrondis :** 10 € répartis entre 3 personnes. Qui paie le centime restant, et comment garantir que la somme des parts égale toujours le total ?
- **Atomicité :** créer une dépense et ses lignes de répartition doit réussir **ou échouer en bloc**. Comment faire avec Supabase ?
- **Suppression :** que devient l'historique des dépenses quand un membre quitte la coloc ?
- **Performance :** que se passe-t-il avec 10 000 messages dans le chat ? Et avec 2 000 dépenses dans l'écran des soldes ?
- **Temps réel :** les évènements reçus sont-ils filtrés par la RLS ? Comment limiter les évènements reçus à **une seule coloc** ?

5. Évaluation

Critère	Points	Ce qu'on regarde
Conception et base de données	4	Schéma cohérent, contraintes, policies complètes, matrice d'autorisations respectée
Sécurité	3	Audit croisé réussi, aucune fuite, pas de clé secrète, RLS comme seule protection
Fonctionnalités	5	Fonctions obligatoires opérationnelles, soldes exacts, temps réel stable
Architecture Flutter	3	Séparation données / interface, widgets réutilisables, pas de duplication, gestion d'état cohérente
Robustesse et qualité	2	Erreurs, chargement, états vides, hors connexion, tests
Démarche	3	Historique Git, journal de bord, `DECISIONS.md` pertinent
Bonus	+1	Une fonctionnalité bonus aboutie